After Deep CFR: four modern poker AIs

Pluribus, ReBeL, Player of Games, AlphaHoldem — a pedagogical tour

Companion volume to *Deep CFR, explained*

PokerTrainer documentation

June 2026

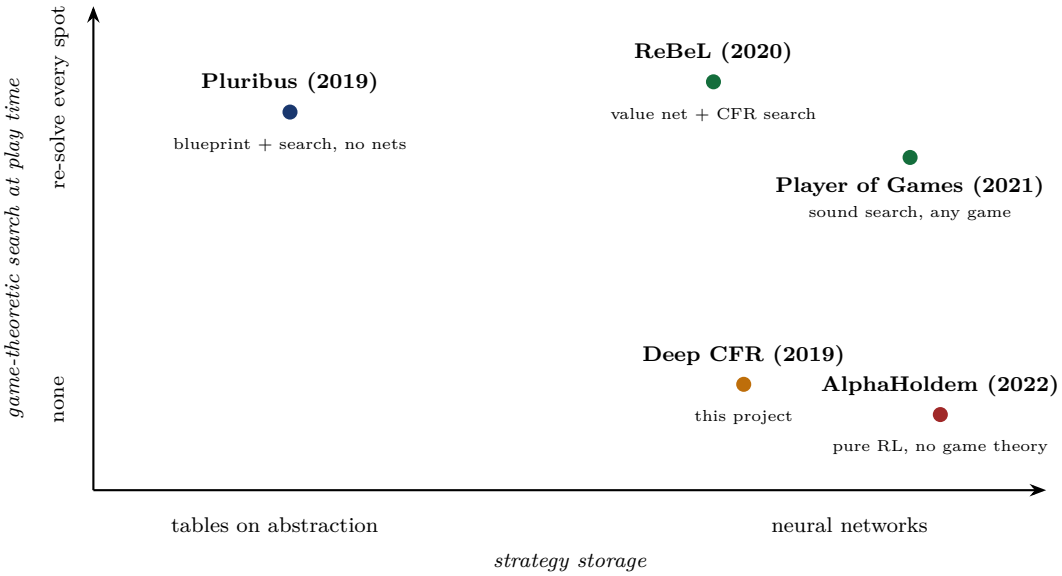
Abstract

The companion tutorial *Deep CFR, explained* covered the theory line from regret matching to Deep CFR. This volume takes the next step and dissects four landmark systems that followed: **Pluribus** (superhuman six-player poker with *no* neural network), **ReBeL** (reinforcement learning + search on *belief states*), **Player of Games** (one sound algorithm for chess, Go *and* poker), and **AlphaHoldem** (end-to-end deep RL, no search at all). For each: the one core idea, how it works mechanically (diagrams + Python pseudo-code), what it achieved, and what it costs. A final section compares all of them — including Deep CFR — on one table and draws lessons for this project.

Contents

1 The design space: one map before four deep dives

Every poker AI answers two independent questions. **(1) Where does game-theoretic reasoning happen?** Offline only (train a strategy, then just play it), or also *online* (re-solve the current situation at the table)? **(2) What stores the strategy?** Tables over abstracted infosets, or neural networks? The four systems in this volume occupy four distinct corners:



Intuition

Read the map as a tug-of-war between two resources: **offline training compute** (bottom row pays it all upfront, then plays instantly) and **online thinking time** (top row keeps solving at the table, which buys precision but costs seconds and engineering). The diagonal from Pluribus to AlphaHoldem is also a diagonal of *decreasing game theory and increasing deep learning.*

1.1 Refresher: the CFR core that three of the four systems share

Pluribus fills its blueprint with it, ReBeL runs it inside every subgame, Player of Games runs its regret-update phase with it — only AlphaHoldem drops it. So here is tabular CFR once, compactly, with its two classic upgrades shown inline (the full pedagogical build-up lives in the companion volume, Sections 2–3):

Listing 1: CFR in one block. `reach[p]` is the probability that player p's own choices lead here; regrets are weighted by the *opponent's* reach, the average strategy by *your own*. The commented lines show the vanilla / CFR$^+$ variants of the update; the active lines are Linear CFR (weight everything by t), which is what Pluribus and Deep CFR use.

```
R, S = tables_of_zeros(), tables_of_zeros()  # regrets / average strategy

def cfr(state, t, reach):
    if state.is_terminal():
        return state.payoffs()                    # (u_0, u_1), zero-sum
    I, p  = infoset(state), state.to_act()
    sigma = regret_matching(R[I])                 # positive part, normalized
    v, v_I = {}, [0.0, 0.0]
    for a in state.legal():
        r = list(reach)
        r[p] *= sigma[a]                          # my move scales MY reach
```

```
        v[a] = cfr(state.child(a), t, r)
        v_I  = [x + sigma[a] * y for x, y in zip(v_I, v[a])]
    for a in state.legal():
        # vanilla CFR:  R[I][a] +=     reach[1-p] * (v[a][p] - v_I[p])
        # CFR+:         R[I][a]  = max(R[I][a] + reach[1-p] * (..), 0)
        R[I][a] += t * reach[1 - p] * (v[a][p] - v_I[p])   # Linear CFR
    S[I] += t * reach[p] * sigma          # the average strategy = the output
    return v_I

def solve(T):
    for t in range(1, T + 1):
        cfr(deal_random_hand(), t, reach=[1.0, 1.0])   # chance sampled
    return normalize(S)                   # converges to Nash (2 players)
```

2 Pluribus: superhuman six-player poker, no neural network

Paper: Brown & Sandholm, *Superhuman AI for multiplayer poker*, Science 2019. **One idea:** a coarse offline "blueprint" strategy plus cheap *depth-limited* online re-solving is enough — even with six players, even with tables instead of networks.

2.1 Why six players changes the rules of the game

Everything in the Deep CFR tutorial relied on the two-player zero-sum correspondence: regret minimization ⇒ Nash, and Nash ⇒ unbeatable. With six players *both* halves break:

> In games with 3+ players, computing a Nash equilibrium is computationally intractable, and — worse — *playing* one is no longer a guarantee: if the others deviate, your equilibrium share can lose. Pluribus therefore drops the theoretical target entirely and aims for an *empirical* one: a strategy that consistently beats elite humans. CFR-style self-play is kept as a *heuristic* that happens to produce strong, balanced play.

2.2 Stage 1 (offline): the blueprint

The blueprint is classic pre-deep-learning machinery, exactly what Deep CFR was designed to replace — and Pluribus shows it is still viable when paired with search:

- **Information abstraction:** strategically similar hands are bucketed (k-means on equity distributions), shrinking 10^{161} situations to a tractable table.
- **Action abstraction:** only a handful of bet sizes exist in the abstract game.
- **Linear MCCFR** (the same external-sampling traversals as our trainer, plus the same linear t-weighting) fills the regret tables — this is exactly the Linear-CFR update of Listing 1, run with sampling.

The famous headline: the blueprint trained in **8 days on a 64-core server**, under 512 GB RAM, **no GPU** — roughly $150 of cloud compute, versus millions of dollars of TPU time for Go-class systems.

2.3 Stage 2 (online): re-solving — but what exactly do you re-solve?

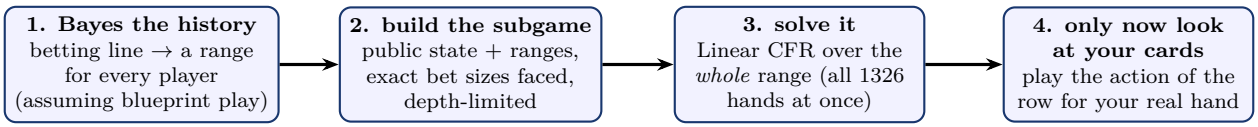
The blueprint alone is too coarse to beat pros (the abstraction blurs exact bet sizes and hand distinctions), so at the table Pluribus computes a fresh, finer strategy for the situation it is actually

in. In chess that sentence is easy to make precise: *search from the current position.* In poker the analogous sentence does not even type-check, and seeing why is most of understanding online re-solving.

Problem 1: the "position" is not a state. What the table shows (board, pot, betting history) is only the *public state*; its value depends on which hands the players might be holding. So the root of a poker subgame must carry, for every player, a *range*: a probability distribution over all 1326 hole-card pairs. And ranges do not fall from the sky — they come from strategies. *If* you assume each player has been following a known strategy so far (the blueprint, plus the outputs of any earlier re-solves), then every public action they took is evidence about their hand, and Bayes' rule turns the betting history into a range: hands that would have folded to the flop raise get probability ~ 0, hands that would have 3-bet get boosted, and so on. Pluribus knows its *own* strategy exactly, so its own range is exact; for opponents it assumes they played approximately the blueprint.

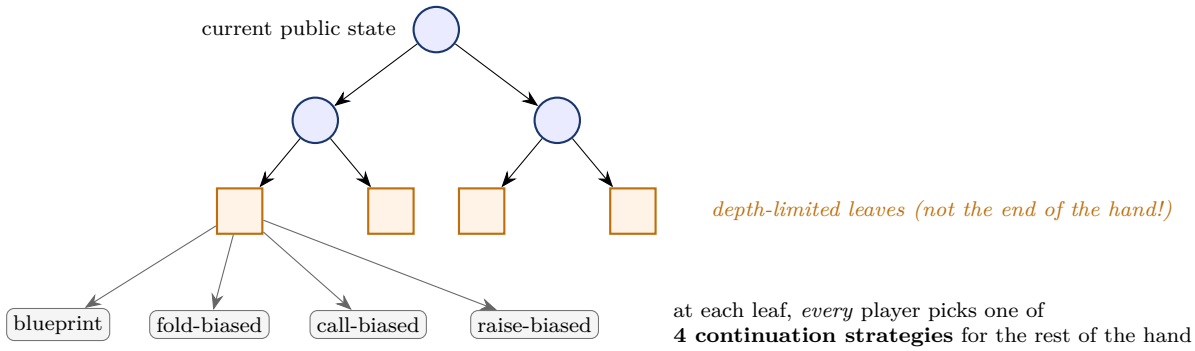
Problem 2: the subgame still reaches too deep. Even rooted on the flop, a no-limit subgame spans turn and river — far too large to solve in seconds. Truncating it at a depth limit needs care, and that is the paper's key invention.

So one online solve looks like this end to end:



2.4 Trick 1: depth limits with continuation strategies

Cutting the subgame off at a depth limit means writing a *number* at each cut point: "the value of arriving here". Any *single* assumption about how play continues below the cut (e.g. "everyone plays the blueprint afterwards") is an assumption the search above will learn to exploit. The fix:



Intuition

Why continuation strategies make shallow leaves honest. If you evaluate a leaf with a *single* fixed policy (say, the blueprint), the search can "cheat": it steers play toward leaves where the blueprint plays badly for the opponent, producing a strategy that only works if the opponent cooperates. Letting every player *choose among several continuations* at the leaf is a miniature game: the value of the leaf is what you can get when the opponent picks their best continuation *against you*. The search can no longer assume a docile opponent — at a tiny fraction of the cost of solving to the end of the hand.

The three biased continuations are cheap to build — take the blueprint, multiply the probability of the biased action (fold / call / raise) by a constant, renormalize — and the value of a leaf under any combination of choices is estimated by rolling the hand out under those strategies. The choices themselves are folded into the CFR solve: each player's leaf-choice is just one more decision in the

subgame, so the equilibrium the solver finds is already robust against all 4^{players} ways play could continue.

2.5 Trick 2: solve for your whole range, not for your hand

A chess engine searches for the best move *in the position it sees*. The poker analogue — "find the best line for the two kings I am holding" — would be a disaster, and understanding why is the other half of online re-solving. Your opponents never observe your hand; they observe your *actions*. If the strategy you compute is a function of your actual hand, your actions become a code for it, and an attentive opponent reads the code: you bet only your strong hands, they fold exactly then, your strong hands stop earning. The whole point of a balanced (GTO) strategy is that the *same* observable action is taken by strong and weak hands in calibrated proportions.

So step 3 of the pipeline solves the subgame **for every hand Pluribus could be holding at once**: the output σ is a whole table — one row per hole-card pair in its range, one action distribution per row — computed jointly so that the *range as a whole* is unexploitable. Only after the solve does Pluribus look at its actual cards and read off that single row (step 4). Bluffs are not a separate mechanism: the equilibrium simply assigns some raising probability to weak rows so that strong rows get paid — balance is an *output* of range-level solving, not a style knob.

Intuition

This also explains an implementation detail: "vector-based" Linear CFR. Since the subgame must be solved for all 1326 hands anyway, the efficient form sweeps the public tree once per iteration carrying a 1326-long *vector* of regrets/values per node, instead of traversing once per hand. Small subgames use this exact vector form; very large ones fall back to sampling (Monte Carlo Linear CFR).

2.6 The full play-time loop

Putting both tricks into the pipeline, here is what happens at a Pluribus decision on, say, the flop:

1. **Freeze the public state**: board, pot, the exact bet it faces (no rounding — the subgame is built with the true size).
2. **Compute ranges** for all players by Bayes from the betting history, assuming blueprint play (its own range: exact).
3. **Build the depth-limited subgame** from this root — typically the rest of the current street plus the start of the next — and attach the four continuation choices at every leaf (Trick 1).
4. **Solve with Linear CFR for the whole range** (Trick 2), a few seconds on two CPUs.
5. **Play**: sample an action from the solved row of its actual hand.
6. **If an opponent later uses a bet size outside the current tree**: build a *new* subgame that contains that exact size and re-solve — *nested* re-solving, repeated as often as needed. (This is how off-tree actions are handled exactly instead of being rounded to the abstraction, the classic leak of blueprint-only bots.)

Listing 2: Pluribus, schematically. The blueprint is Deep CFR's ancestor (same traversals, tables instead of nets); play time adds range-aware, depth-limited re-solving.

```
def blueprint(T):                          # OFFLINE, once, ~8 days on CPUs
    for t in range(1, T + 1):
        for p in range(6):                 # six players!
            mccfr_traverse(deal(), p, t)   # external sampling, linear weights
```

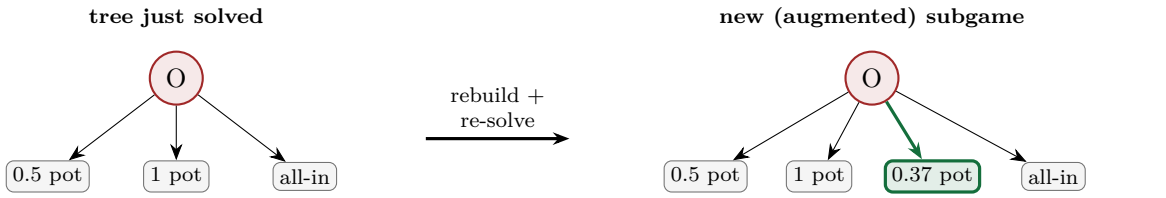
```
    return regret_tables                # indexed by ABSTRACT infosets

def resolve(public_state, ranges):      # ONLINE: one subgame solve
    G = subgame(public_state, ranges,             # root carries the RANGES
                depth_limit=rest_of_street_plus,
                bet_sizes=exact_bets_faced)       # no rounding
    for leaf in G.leaves():             # Trick 1: 4-way continuation choice
        leaf.attach_choices([blueprint, fold_bias, call_bias, raise_bias])
    return linear_cfr(G)                # Trick 2: sigma[hand] for ALL hands

def act(history, board, my_hand):
    if preflop and on_tree(history):    # preflop abstraction is dense:
        return sample(blueprint[abstract(history, my_hand)])
    ranges = bayes_ranges(history, board, assumed=blueprint)
    sigma  = resolve(public_state(history, board), ranges)
    return sample(sigma[my_hand])       # look at MY cards only at the end
# off-tree opponent bet later? build a new subgame with that exact size
# and call resolve() again (nested re-solving).
```

2.7 Close-up: the two questions everyone asks

Q1 — an opponent bets a size that is not in the tree. What happens, mechanically? Suppose the subgame Pluribus just solved offered the opponent three sizes at some node — {0.5 pot, 1 pot, all-in} — and the human bets 0.37 pot. The solution Pluribus is holding has *no entry* for what just happened. The historical fix ("action translation": round 0.37 to the nearest size and pretend) is exactly the leak humans learned to attack by betting *between* the abstraction points. Pluribus instead rebuilds:



human bets **0.37 pot**: no branch!

the new equilibrium decides *which hands* bet 0.37 pot
→ posterior range comes from the solve itself, no rounding

1. **Rebuild**: a fresh subgame rooted just before the bet, with the opponent's action set at that node *augmented* by the exact 0.37-pot size.
2. **Ranges at the new root** are taken from *before* the off-tree action — everything up to that moment was on-model, so Bayes is still well-defined.
3. **Re-solve** the augmented subgame as usual (depth limit, continuation leaves, whole-range Linear CFR).
4. The subtle part: the new equilibrium contains **a strategy for the opponent at the augmented node** — it computes *which hands would bet* 0.37 *pot at equilibrium*. Bayes against *that* computed strategy gives the range going forward. The off-tree action is never mapped to a fiction; the solver *invents its correct interpretation* as part of the equilibrium.
5. Another off-tree size on a later street? Do it all again — hence *nested* re-solving.

Intuition

The asymmetry to remember: Pluribus's *own* bet sizes can stay few — it chooses them. It is the *opponent's* freedom that makes the full game impossible to pre-solve. Nested re-solving prices each opponent choice *at the moment it happens* instead of pre-computing responses to a continuum — the same "pay compute only for situations that actually arise" logic that motivates search in the first place. (Preflop, where Pluribus plays the blueprint directly, the action abstraction is dense enough that rounding a rare off-tree size costs little.)

Q2 — the ranges assume opponents follow the blueprint. What if they just… don't? Separate three cases.

Case 1: the deviation is an off-tree action. That is Q1 — structural, fully handled; the augmented equilibrium even defines what the new action "means".

Case 2: on-tree actions, wrong frequencies. The human raises twice as often as the blueprint would. Pluribus's posterior range for them is then simply *wrong* — it credits the line with blueprint frequencies. And Pluribus does *nothing* about it: by deliberate design it has no opponent modeling and never adapts to observed tendencies. Partly robustness (an adapting bot can be manipulated by feigned tendencies for later betrayal), partly principle: its defense is **balance, not mind-reading**. A range is a statement about a *model*, not about the human — Bayes answers "which hands would have taken this line *if* the opponent played the assumed strategy".

Case 3: so why is playing on-model still sound? Zero-sum equilibrium logic: the strategy Pluribus computes is (approximately) unexploitable *within the model game*, and an opponent playing off-model frequencies is, from the model's viewpoint, deviating from equilibrium — deviations do not gain in expectation. The model's mixed strategies also keep ranges from ever collapsing to zero on the human's true holding (regret matching leaves small probability on many actions), and every nested re-solve refreshes the range model. The honest catch is the next box.

Watch out

Where the soundness leak is. An opponent can deviate *early*, accept a small on-model loss, *poison the assumed range*, and harvest a larger error from the re-solver later — this is exactly what makes naive ("unsafe") re-solving theoretically attackable in two-player zero-sum. Libratus closed the hole with *safe* re-solving gadgets that guarantee the re-solved strategy never does worse than the blueprint against any opponent hand. Pluribus — playing six-max, where equilibrium guarantees were abandoned anyway (Sec. 2.1) — accepts the heuristic; empirically, five elite professionals over 10 000 hands did not find a way in. One more instance of the multiplayer trade: theory exchanged for scale, validated at the table.

Results. Against elite professionals (including World Series champions), in both 5 humans + 1 AI and 1 human + 5 copies formats, Pluribus won decisively. Each decision takes seconds on two CPUs.

Watch out

Pluribus is often summarized as "deep learning beats poker pros". It contains **no neural network at all**. Its lesson is the opposite of the hype: *search and good abstractions are devastatingly cost-effective.* When you read a new-method paper, always ask what a tuned blueprint-plus-search baseline would have done.

3 ReBeL: reinforcement learning + search on belief states

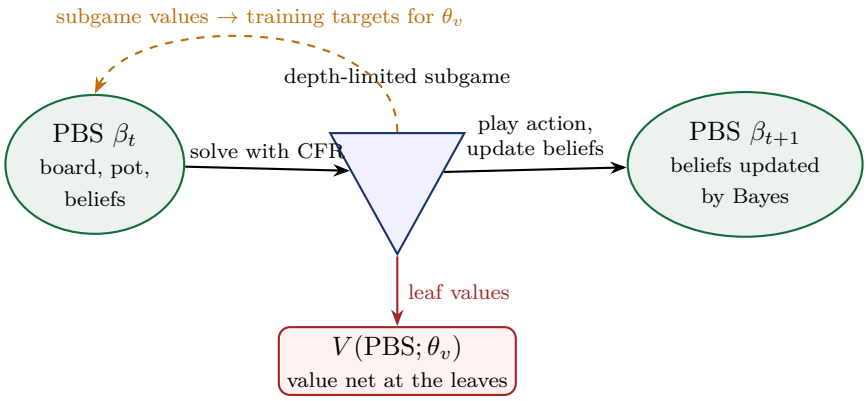
Paper: Brown, Bakhtin, Lerer, Gong, *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games*, NeurIPS 2020. **One idea:** an imperfect-information game becomes a *perfect-information* game if the "state" is taken to be the **public belief state** — then

AlphaZero-style self-play + search applies, with CFR doing the searching.

3.1 The conversion trick

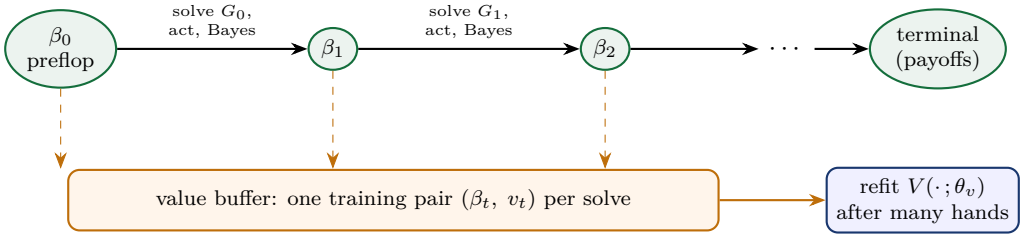
Public belief state (PBS). Everything publicly visible (board, pot, betting history) *plus*, for each player, the probability distribution over their possible private hands, as computable by an outside observer who knows both players' strategies. In hold'em each belief is a vector over the 1326 possible hole-card pairs.

Why this works: if both players' policies are common knowledge, then Bayes' rule updates the beliefs after every public action — so the PBS evolves *deterministically given the policies*, like a board position. Hidden information has been traded for a bigger, continuous state. Values become well-defined functions of the PBS, so a neural network $V(\text{PBS})$ can be trained the way AlphaZero trains its value net.



3.2 The self-play loop, one hand at a time

The loop is easiest to follow as a single hand of self-play, narrated. First, the cast: ReBeL plays *both* seats. At the start of an episode a real deal hands private cards to each seat (the *world state*); on top of that, the algorithm maintains one shared bookkeeping object, the PBS — which contains no secrets (board, pot, two belief vectors), so it looks identical from both seats. The hand then alternates between *solving* (a fresh subgame at the current PBS) and *stepping* (an action is played, beliefs update, possibly new board cards arrive):



At each decision point, six things happen, in order:

1. **Freeze the current PBS** β_t: board, pot, and both belief vectors as they stand.
2. **Build the depth-limited subgame** G_t rooted at β_t — roughly the rest of the current betting round; wherever new cards would be dealt, the tree stops and the value net $V(\cdot\,;\theta_v)$ will answer.
3. **Solve it**: T_{cfr} iterations of plain *tabular* CFR inside G_t (this is where volume 1's algorithm literally runs, thousands of times per hand). The policy net θ_π warm-starts the iterations so fewer are needed; every leaf evaluation is a query to the value net.

4. **Store a training pair**: (β_t, v_t), where v_t is the per-hand value vector of the root under the *average* of the T_{cfr} policies — i.e. "what this PBS is worth under near-equilibrium play". This is the label θ_v will later regress on, exactly like AlphaZero labels positions with search outcomes.
5. **Choose what to actually play**: pick *one random iteration* $t' \sim \text{uniform}(1..T_{\text{cfr}})$ and use its policy $\sigma_{t'}$ — *not* the average. This is the strangest-looking choice in the method; both reasons (training-distribution consistency and safety-by-randomization) are unpacked in the close-up, Q2 below.
6. **Act and update**: the seat to act looks at its real cards and samples its row of $\sigma_{t'}$; the action is public; Bayes' rule pushes *both* belief vectors through $\sigma_{t'}$, giving β_{t+1} (if a street ends, the dealt board cards update the PBS too).

When the hand reaches a terminal PBS, the episode's pairs join the buffer; after many hands, θ_v is refit on the buffer and θ_π on the stored search policies. **At test time against a human, the identical loop runs minus the stores** — search-with-net-leaves *is* the player; training merely makes the net worth searching with.

Listing 3: ReBeL's training loop — the six steps above, in code. The same machinery runs at test time, minus the buffer writes.

```
def rebel_episode(theta_v, theta_pi, buffer):
    deal = sample_world()                    # real cards for both seats
    beta = initial_pbs()                     # (1) blinds, uniform beliefs
    while not beta.is_terminal():
        G = depth_limited_subgame(beta)      # (2) leaves -> V(.; theta_v)
        policies = run_cfr(G, T_cfr,         # (3) tabular CFR inside G,
                           warm_start=theta_pi)
        buffer.add(beta,                     # (4) value target: root values
                   value_of_root(G, average(policies)))  # of the AVERAGE
        sigma = policies[randint(1, T_cfr)]  # (5) play a RANDOM iterate
                                             #     (why: close-up, Q2)
        hand = deal[beta.to_act()]
        a = sample(sigma[hand])              # (6) act with the real cards...
        beta = bayes_update(beta, sigma, a)  #     ...Bayes BOTH beliefs
    refit(theta_v, buffer)                   # after many hands
```

Intuition

Where Deep CFR and ReBeL put the neural network. Deep CFR's nets *replace the regret tables* inside one global solve. ReBeL's net replaces *the bottom of the search tree*: CFR itself stays tabular but only ever sees a small window of the game, and the net summarizes everything below the window. That is exactly AlphaZero's division of labor, transplanted to imperfect information — which is why the paper's title says "RL + Search".

3.3 What the value net actually sees and says

It is worth being concrete, because the input/output shape *is* the method. The input is the whole PBS: the public features (board cards, pot, bets) *plus both belief vectors* — ∼1326 probabilities per player. The output is **not one number**: it is **a value per hand, for each player** (the *infostate values*). It has to be: the tabular CFR running above the leaves updates a regret *per hand* — "what is arriving at this leaf worth *if I hold AA*? *if I hold 76s*?" — so a single scalar would be useless to it. Equivalently: the net is a learned generalization of exactly the per-hand counterfactual values that DeepStack pioneered, with the beliefs as part of the input so one net covers every range shape.

The subgame ReBeL solves spans roughly the rest of the current betting round; wherever new public cards would be revealed, the tree stops and the net answers. Solving the subgame with T_{cfr} tabular

CFR iterations queries the net thousands of times — this is why the net must be fast, and why its accuracy directly caps the quality of play (the $\varepsilon_{\text{approx}}$ floor of volume 3).

3.4 Close-up: the two questions everyone asks

Q1 — beliefs are computed from the players' strategies. How can ReBeL know the *opponent's* beliefs? It cannot, and it does not pretend to: like Pluribus's ranges, the PBS is a statement about a *model*. ReBeL computes *all* beliefs — its own and its opponent's — from the assumption that both players follow the policies its own procedure generates (this is the "common knowledge" assumption in the paper). Against an opponent who actually plays differently, the on-model logic of Pluribus's Q2 applies verbatim: deviations are, within the model, unprofitable; early deviations can poison the beliefs; and the next question is precisely ReBeL's elegant answer to that poisoning attack.

Q2 — after solving the subgame, why play the strategy of a *randomly sampled* CFR iteration rather than the final average? This is the strangest-looking line of the pseudo-code and the paper's quiet masterstroke; it solves two problems at once.

- *Training consistency.* The value targets stored for the net are values of the subgame's *average* policy — an average over all T_{cfr} iterations. If play (and hence the belief updates that produce the *next* PBS) always followed one fixed choice, the distribution of next-PBSs would not match the distribution the stored values implicitly assume, and the net would be trained on one distribution and queried on another — compounding off-distribution error down the game. Sampling the iteration uniformly makes "how beliefs are generated" agree, in expectation, with "how values were averaged".
- *Safety.* Recall the hole in naive re-solving (Pluribus close-up, Q2): an opponent who knows how you re-solve can poison your beliefs. ReBeL's theorem says that playing a uniformly sampled CFR iterate makes the depth-limited search *provably* low-exploitability — the same guarantee Libratus needed hand-built gadget games for, obtained instead by randomization. Intuition: the opponent cannot tailor an exploit to "the" strategy at the subgame root, because there is no single such strategy — they face the whole cloud of CFR iterates, and beating the cloud on average is exactly what CFR's equilibrium guarantee forbids.

Intuition

A useful contrast triangle. **Libratus**: safety by *construction* (gadget subgames). **Pluribus**: safety *dropped* (six-max, empirical validation). **ReBeL**: safety by *randomization* (sample an iterate). All three are answers to the same question — "what stops an opponent from gaming my re-solver?" — and the three answers neatly span engineering, pragmatism, and theory.

Results. Superhuman heads-up no-limit hold'em with far less poker knowledge than DeepStack-/Libratus: it beat top HUNL professional Dong Kim by **165 mbb/hand over 7 500 hands** (Kim was the best-performing human of the Libratus match). Also solid play in Liar's Dice, showing generality. The experiments used a fixed menu of bet sizes for tractability; handling off-tree sizes by nested re-solving (as in Libratus/Pluribus) is compatible but orthogonal.

Watch out

The PBS is the whole trick — and the whole bill. The belief vector needs one probability per private state: 1326 in hold'em is fine; a game with 10^{10} private states is not. And beliefs are only well-defined if the policy they are computed from is *common knowledge* — ReBeL effectively "announces" its strategy, which is safe at equilibrium but subtle to reason about away from it. Porting ReBeL to a new game starts with the question: *can I even write down the belief vector?*

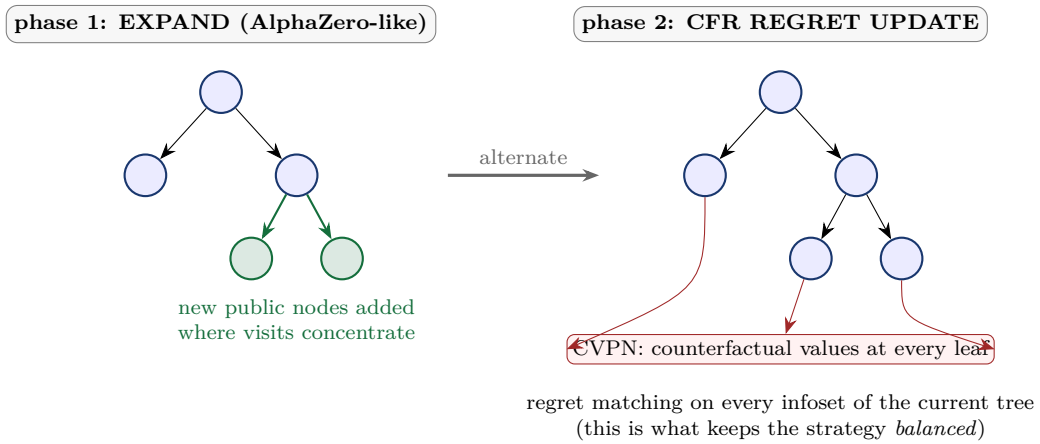
4 Player of Games: one sound algorithm for chess, Go and poker

Paper: Schmid et al., *Player of Games*, 2021; published in Science Advances 2023 under the name *Student of Games*. **One idea:** merge AlphaZero's *growing* search tree with CFR's *sound* handling of hidden information, so the same agent plays both kinds of games.

4.1 What "sound" buys you

AlphaZero's MCTS is built for perfect information; applied naively to poker it converges to exploitable strategies (it has no concept of balancing a range). ReBeL is sound but poker-shaped. Player of Games (PoG) asks: what is the *minimal* unification? Its answer has two components:

- a **counterfactual value-and-policy network (CVPN)**: given a public state plus beliefs (like ReBeL's PBS), it outputs *per-infoset counterfactual values* for both players and a policy prior. In a perfect-information game the beliefs are trivial and it degrades exactly to AlphaZero's value+policy net.
- **growing-tree CFR (GT-CFR)**: the search keeps a small *public* tree and alternates two phases — *expand* the tree where the policy wants to visit (AlphaZero-style), then run *CFR regret updates* over the whole current tree with CVPN values at the leaves.



Listing 4: GT-CFR and sound self-play (schematic).

```
def gt_cfr(beliefs, cvpn, n_rounds):
    tree = PublicTree(root=beliefs)            # starts tiny
    for _ in range(n_rounds):
        for _ in range(E):                     #  EXPAND
            leaf = descend_by_policy_and_visits(tree)
            tree.add_children(leaf, prior=cvpn.policy(leaf))
        for _ in range(R):                     #  REGRET UPDATE
            cfr_iteration(tree, leaf_values=cvpn.values)
    return tree.root_policy(), tree.root_values()

def sound_self_play(cvpn):
    while True:
        traj = play_game(act=lambda b: gt_cfr(b, cvpn, N))
        # every position where the net was queried at a leaf goes into a
        # QUERY BUFFER; a background process re-solves those positions
        # (recursively, with more search) to produce improved value targets
        train(cvpn, value_targets_from(query_buffer), policy_targets_from(traj))
```

Intuition

The query buffer is the conceptual heart of the training. The net is trained *at exactly the places where the search trusted it* — each leaf query becomes a promise to go back later, search deeper from that spot, and return with a better value to learn from. Compare our project's stage-1 oracle: the principle "validate the function where you actually rely on it" is the same; PoG just turns it into the training signal itself.

4.2 What a node of the search tree actually is

Same exercise as for ReBeL, because here too the data shapes *are* the method. A node of PoG's tree is a *public* state **plus the players' belief vectors** — a PBS in ReBeL's vocabulary. The CVPN takes that in and emits two things *per hand*: a counterfactual value (used when the node is a search leaf) and a policy prior (used to decide where to grow the tree, exactly like AlphaZero's policy head guides PUCT). Now run the degeneration test: in chess, the "belief vectors" are trivial (one world state, probability 1), the public tree *is* the game tree, the counterfactual values collapse to ordinary state values — and PoG's machinery reduces to AlphaZero's value-plus-policy search. The imperfect-information generality costs nothing when there is nothing to be uncertain about. That is the precise sense in which it is "one algorithm for both kinds of games".

The training loop runs two processes. *Actors* play games with a modest search budget, logging every leaf position where the CVPN was queried into the **query buffer**. A *trainer* pops queried positions and re-solves them with a *larger* budget, producing better values than the net originally returned — those become its regression targets (and the deeper solves may query new leaves, which re-enter the buffer with decaying probability: a self-improvement ladder in which the network forever chases its own deeper searches).

4.3 Close-up: the two questions everyone asks

Q1 — ReBeL also searches over belief states. What is actually different? The *shape of the searched region.* At every decision, ReBeL builds the **full, dense** subgame down to its depth limit and solves all of it with tabular CFR — affordable in poker, where a betting round has a handful of public actions, but hopeless in games with large public branching or long horizons between chance events. PoG instead **grows a sparse tree** one node at a time, AlphaZero-style: the policy prior and visit counts decide which few branches deserve nodes at all, and the CFR phase keeps whatever tree currently exists balanced. Sparse growth is what lets the same code play chess (huge branching, no beliefs) and Scotland Yard (long imperfect-information horizons) — and it is why PoG, not ReBeL, is the one that generalized beyond poker.

Q2 — why does plain AlphaZero/MCTS fail at poker in the first place? Because MCTS is an *argmax engine*: as visits accumulate, it concentrates on the single best-looking action, and its play at a state becomes essentially deterministic. In perfect information that is exactly right — some move *is* best. In poker, determinism is a tell. A deterministic policy at a bluffing spot either always bluffs or never bluffs, and either way an observant opponent prints money (fold always / call always). Equilibria in imperfect information are *mixed* — calibrated frequencies, not best moves — and producing mixtures is precisely what regret matching does and tree-search argmax does not. GT-CFR is therefore not a hybrid for hybrid's sake: the expansion phase brings MCTS's scalability, the regret phase brings CFR's balance, and each compensates the other's failure mode.

Intuition

One sentence to keep: *AlphaZero asks "what is the best move here?"; CFR asks "what frequencies keep me unexploitable here?"* — and Player of Games is the observation that you can ask the first question about *where to look* while asking the second about *what to play.*

Results. One set of weights per game, same algorithm: expert-level chess and Go (below a specialized AlphaZero at equal compute — the price of generality), beats Slumbot (the strongest open heads-up poker bot) and is not exploited by a local best-response probe, and defeats the state of the art in Scotland Yard, a hide-and-seek board game with imperfect information.

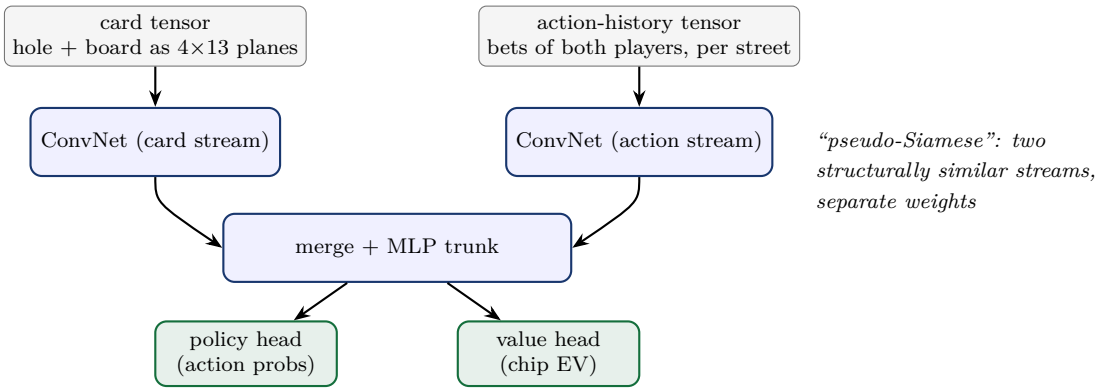
Watch out

Soundness here is a statement about the *search*, given good values: more search cannot make play more exploitable. It is not a global optimality proof, and in perfect-information games generality costs head-to-head strength versus a specialist. "One algorithm for everything" and "the best algorithm for each thing" remain different targets.

5 AlphaHoldem: what if we drop the game theory entirely?

Paper: Zhao, Yan, Li, Li, Xing, *AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning*, AAAI 2022. **One idea:** no CFR, no search, no beliefs — a single network maps the observation directly to an action, trained with a carefully stabilized PPO against a pool of past selves.

5.1 Architecture: two eyes, one brain



Note what is *absent*: no belief vector, no infoset enumeration, no regret anything. The action history tensor plays the role our token sequence plays in this project — it is the only channel through which the network can infer what the opponent's range looks like.

Why these particular encodings? The card stream stacks 4×13 binary planes (suit × rank), one group per street: hole cards, flop, turn, river. On that grid, poker structure is *geometric*: a straight draw is a run of adjacent *columns*, a flush draw is mass concentrated in one *row*, a paired board is two cells sharing a column — exactly the local patterns small convolutions detect cheaply. The action stream encodes, per betting round and per player, which action was taken and the normalized bet size, again as planes, so the same convolutional machinery picks up betting *patterns* (e.g. "raise–raise" textures) rather than treating the history as an opaque flat vector — the same lesson, in convolutional form, that pushed this project from a flat 816-dim input to a token sequence.

5.2 Making PPO survive poker

Vanilla PPO collapses on poker: rewards span ±200 big blinds with brutal variance, and importance ratios explode on rare lines. AlphaHoldem's **Trinal-Clip PPO** adds two clamps to the standard clipped loss:

Listing 5: Trinal-Clip PPO (the three clips). delta1 caps the policy ratio when the advantage is negative; delta2/delta3 cap the value target at what is physically winnable/losable in the hand.

```
def trinal_clip_losses(ratio, adv, v_pred, v_target, eps, d1, d2, d3):
    # 1+2) standard PPO clip, PLUS an upper cap d1 (> 1+eps) on the ratio
    #      when adv < 0 -- a huge ratio times a negative advantage otherwise
    #      yields an unbounded, variance-exploding policy loss
    r = clip(ratio, 1 - eps, 1 + eps) if adv >= 0 else \
        clip(ratio, 1 - eps, d1)
    policy_loss = -min(ratio * adv, r * adv)
    # 3) value target clipped to the chips actually reachable this hand
    #    (d2 = max losable, d3 = max winnable -- pot/stack dependent)
    value_loss = (v_pred - clip(v_target, -d2, d3)) ** 2
    return policy_loss, value_loss
```

5.3 K-best self-play against strategy cycling

Naive self-play (always train against the latest self) famously *cycles* in poker: agent B beats A, C beats B, and C loses to A again — rock-paper-scissors in strategy space, no monotone progress. AlphaHoldem trains each new agent against the K **best historical versions** simultaneously, scored by an Elo-like rating; a new agent enters the pool only if it earns its place. (AlphaStar's league is the same idea at larger scale.)

Listing 6: K-best self-play.

```
pool = [random_init()]
while training:
    learner = clone(best(pool))
    for _ in range(n_steps):
        opponent = sample(pool)                 # mix over the K best
        rollouts = play(learner, opponent)
        update(learner, trinal_clip_ppo(rollouts))
    pool = top_k(pool + [learner], K, by=elo)   # survival of the fittest
```

5.4 Close-up: the two questions everyone asks

Q1 — no search, no ranges: where does balance (bluffing) come from, if anywhere? Only from gradient pressure. If the current policy is predictable — bets only its strong hands — then *provided some opponent in the K-best pool punishes that pattern*, those rollouts lose money and the policy gradient pushes toward mixing. Balance can emerge, but it is an *equilibrium of the pool*, not of the game: nothing certifies that the pool contains the punisher for every leak. Contrast the other systems in this volume: Pluribus *computes* balance per spot (Trick 2, the whole-range solve); AlphaHoldem *hopes SGD finds it*. This is the precise content of the "no certificate" trade — and why follow-up work exploiting pure-RL poker agents keeps finding leaks that self-play ratings never showed.

Q2 — why does naive self-play cycle, and why does a K-best pool help? Volume 3 proved the deep reason: in zero-sum games, the *current* strategies of self-play never converge — they orbit the equilibrium forever (the RPS table showed a 73%-Paper current strategy mid-convergence); only the *average* converges. CFR is built around that fact: it carries the average explicitly. A vanilla RL agent keeps only a current policy, so it *lives* the orbit instead of averaging it: B learns to beat A, C learns to beat B, and C loses to A — rock–paper–scissors in strategy space, with ratings that go up forever while true strength goes nowhere. Training against the K best *historical* agents at once is a crude integral over the orbit: to enter the pool you must do well against a *spread* of past play styles simultaneously, which damps the cycle — a poor man's strategy averaging. (AlphaStar's

league is the same medicine at industrial scale.) Seen this way, K-best self-play is not an RL hack; it is CFR's averaging theorem, smuggled back in through the opponent distribution.

Intuition

The two clips of Trinal-Clip PPO have the same shape as this project's regret-scale reasoning: bound every learning target by what is *physically reachable in the hand* (± the committed chips). Heavy tails in poker are not noise to average away — they are impossible targets to refuse.

Results. Over 100 000 evaluation hands, AlphaHoldem beats Slumbot and a DeepStack re-implementation, after **3 days of training on one 8-GPU server**, and then decides in **2.9 ms** on a single GPU — roughly a thousand times faster than search-based systems.

Watch out

No theory, no certificate. Nothing bounds AlphaHoldem's exploitability; "beats Slumbot" does not imply "cannot be exploited by a strategy that hunts for its leaks" — and follow-up work has indeed found pure-RL poker agents exploitable. This is the precise trade this corner of the design-space map makes: spectacular speed and simplicity, no safety net. Our project's self-play collapses (the 1M-step incoherent policy) were a small taste of the same phenomenon — and our stage-1 Nash oracle is exactly the kind of certificate AlphaHoldem lacks.

6 Side by side, and what this project takes from each

	Deep CFR	Pluribus	ReBeL	PoG/SoG	AlphaHoldem
year	2019	2019	2020	2021	2022
core engine	CFR	CFR	CFR+RL	CFR+MCTS	PPO
neural nets	value+policy	**none**	value+policy	CVPN	policy+value
search at play time	no	yes	yes	yes	no
belief states needed	no	no	yes	yes	no
abstraction needed	no	yes	no	no	no
players	2	**6**	2	2	2
theoretical target	Nash	none (empirical)	Nash	sound search	none
training hardware	GPUs	64 CPU cores, 8 days	many GPUs	TPUs	8 GPUs, 3 days
decision latency	ms	seconds	seconds	seconds	**2.9 ms**

Intuition

Choosing for this project. PokerTrainer sits in the Deep CFR cell on purpose: no abstraction to hand-craft (the token transformer reads raw infosets), no play-time search infrastructure (one network ships in the Flutter inspector and the C ABI), and a real theoretical target (Nash) that gives us measurable progress — which the stage-1 push/fold oracle exploits directly. The map's other corners tell us what we are giving up: Pluribus-style re-solving would buy precision at the cost of an online solver; ReBeL/PoG belief machinery would buy soundness-at-depth at the cost of 1326-dim belief plumbing; AlphaHoldem's pure RL would buy simplicity at the cost of the very guarantees we built the oracle to enforce.

Concrete techniques worth borrowing here, in increasing order of effort:

1. **K-best opponent pools (AlphaHoldem)** — our evaluation matches already revealed self-play drift; rating checkpoints with Elo and evaluating new ones against the K best is cheap and directly transplantable.

2. **Reward/value clipping by reachable chips (AlphaHoldem)** — the same physical bounds (± effective stack) we used to reason about regret scale apply to any value-learning component we add.
3. **Continuation-strategy leaf evaluation (Pluribus)** — if we ever add depth limits beyond the current crude bootstrap (flagged in `cfr_coro.py`), the 4-continuation construction is the principled upgrade.
4. **Query-buffer-style targeted validation (PoG)** — extend the stage-1 oracle idea: log the infosets where the AdvNet's regrets most influenced traversals, and audit/validate exactly there.

7 Further reading

- Brown, Sandholm (2019). *Superhuman AI for multiplayer poker* (Pluribus). Science.
- Brown, Sandholm (2018). *Depth-Limited Solving for Imperfect-Information Games* — the continuation-strategies construction Pluribus builds on.
- Brown, Bakhtin, Lerer, Gong (2020). *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games* (ReBeL). NeurIPS.
- Schmid et al. (2021/2023). *Player of Games / Student of Games: A unified learning algorithm for both perfect and imperfect information games.* Science Advances.
- Zhao, Yan, Li, Li, Xing (2022). *AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning.* AAAI.
- Moravčík et al. (2017). *DeepStack* — the origin of learned counterfactual values at depth limits, ancestor of both ReBeL and PoG.
- Companion volume: *Deep CFR, explained* (`docs/deep_cfr_tutorial/`) — regret matching, CFR, MCCFR, Deep CFR, and this project's curriculum.